Putting Your Helpers in a Module

As with any other Ruby class, you can also define helper methods in a separate module and include them in your example groups. For instance, here's the outline of the acceptance specs you wrote in *Starting On the Outside: Acceptance Specs*:

07-structuring-code-examples/07/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
RSpec.describe 'Expense Tracker API', :db do
  include Rack::Test::Methods

  def app
    ExpenseTracker::API.new
  end

  # ...
end
```

As you define specs in new files that drive the API, you'll find yourself using rack-test in multiple places. If you put this bit of glue code in a module, you can easily include it into all your acceptance specs:

07-structuring-code-examples/08/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
module APIHelpers
  include Rack::Test::Methods

  def app
    ExpenseTracker::API.new
  end
end

RSpec.describe 'Expense Tracker API', :db do
  include APIHelpers

  # ...
end
```

Because of the added layer of indirection, we recommend using a module only if you need to use the same helper methods in more than one example group.

Including Modules Automatically

Even a one-liner like include APIHelpers is easy to forget. If you need to include the same helper module into many or all of your specs, you can call config.include in an RSpec.configure block:

07-structuring-code-examples/08/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
RSpec.configure do |config|
  config.include APIHelpers
end
```